Agile Software and DevOps Lab

Mini Project: ShopSphere E-Commerce Platform

Experiments 1 to 7: Member-wise Work Division and Detailed Steps

Group of Three Members:

- **Member A (Project Lead / Foundation / Owner):** Shreya Khandelwal
- **Member B (Builder):** Khushboo
- **Member C (Integrator):** Mahek

---

1. Introduction

This document provides the member-wise work division and step-by-step instructions for **Experiments 1 to 7** of the Agile Software and DevOps Lab, implemented on a full-stack group mini-project: **ShopSphere E-Commerce Platform**.

The steps, commands, and workflows strictly adhere to the academic laboratory syllabus. Lab-specific names, repository identifiers, module names, job names, and IP addresses are customized for the **ShopSphere E-Commerce** project.

The objective of this mini-project is to demonstrate a complete, production-grade end-to-end DevOps lifecycle: $$\text{Jira (Plan)} \longrightarrow \text{Git & GitHub (Code)} \longrightarrow \text{Docker & Compose (Package & Run)} \longrightarrow \text{Jenkins (Build, Test, Deploy)} \longrightarrow \text{Nagios / Prometheus (Monitor)}$$

Every team member performs an active, verified role in each experiment.

1.1 Experiments Covered

Experiment File	Topic	Syllabus Aim
EXP 01	**Git**	Version control operations using Git (35 distinct commands & workflows)
EXP 02	**Docker**	To containerise the application using Docker (25 Docker commands)
EXP 03	**Docker Compose**	To deploy multi-service applications using Docker Compose
EXP 04	**Jenkins CI**	Understand Continuous Integration, install Jenkins, build Declarative and Scripted pipelines
EXP 05	**Jenkins + GitHub**	To automate application deployment using GitHub / Jenkins
EXP 06	**Jira**	Agile Software Project lifecycle using Jira with DevOps Pipeline Integration (12 steps)
EXP 07	**Monitoring**	Implement monitoring using Prometheus / Grafana / Nagios (9 steps)

1.2 What the Application Provides

- **Application Architecture:** Full-stack architecture with backend REST API running on port `5000` (`http://localhost:5000`) and frontend shopping portal on port `3000`.
- **Health Endpoint:** A dedicated route `GET /health` returning HTTP status `200` (`{"status":"UP","database":"CONNECTED"}`).
- **Dependency Files:** `requirements.txt` / `package.json` maintaining runtime packages. Test frameworks (`pytest` / `jest`) are added dynamically in EXP 06 to demonstrate realistic branch merge conflicts.

1.3 Names and Placeholders Used

Item	Value Used in This Document
GitHub Repository	`shopsphere-ecommerce` (Unified repository for EXP 01, 05, and 06)
Project Folder	`devops-fullstack-app`
`<gh-user>`	GitHub username of **Member A (Shreya Khandelwal)** (Repo Owner)
Git Feature Branches (EXP 01)	`feature/khushboo-product-reviews`, `feature/mahek-discounts-coupons`
Jenkins Jobs	`ShopSphere-Declarative`, `ShopSphere-Scripted`, `ShopSphere-Demo` (EXP 05)
Jira Project	`ShopSphere Team` (Issue keys: `SHOP-101`, `SHOP-102`, `SHOP-201`, `SHOP-202`)
Compose Folder	`compose-demo` (inside repository root)
Monitoring Host Name	`ShopSphere_Host_001`

1.4 Ports and Network Topology

Service	Port	Host PC Location
ShopSphere Backend API	`5000`	Each member's own PC / Container
ShopSphere Frontend Web	`3000`	Each member's own PC / Container
Jenkins Automation Server	`8080`	**Member A's PC (Shreya)** (Members B & C access over LAN)
Compose Nginx Reverse Proxy	`8081`	Each member's own PC
Compose phpMyAdmin / Database	`8082` / `5432`	Each member's own PC
Nagios Web Interface	`8090`	**Member A's PC (Shreya)** (Mapped to 8090 to avoid conflict with Jenkins 8080)
Prometheus Metrics Scraper	`9090`	Member A's PC / Docker container
Grafana Dashboard	`3001`	Member A's PC / Docker container
NSClient++ (NRPE Agent)	`5666`	**Member B's PC (Khushboo)** (Monitored remote Windows host)

1.5 Group Operating Rules

1. All command-line instructions are written for Windows PowerShell / Bash.
2. Every member works on their own workstation with their own GitHub account and captures individual command execution outputs for their report.
3. Steps must be performed in exact numerical sequence. Where a member must wait for a peer, it is explicitly flagged as **Hand-off**.
4. Team synchronization steps (e.g. `git clone`, `git pull`, `git push origin main`) are marked as **Team step (Collaboration workflow)**.

2. Member Roles

Member	Role	Key Responsibilities
Member A (Shreya Khandelwal)	**Foundation / Owner**	Creates initial Git project, GitHub repository, Jira project space, Jenkins server, and Nagios/Prometheus monitoring server. Members B and C rely on A's initial push and invitations.
Member B (Khushboo)	**Builder**	Develops first feature branch (`reviews`), Docker commands 10–17, Compose database services, Declarative Jenkins pipeline, Jira smart commit, and remote Nagios agent.
Member C (Mahek)	**Integrator**	Develops second feature branch (`discounts`), resolves merge conflict, Docker commands 18–25, Compose redis and lifecycle commands, Scripted Jenkins pipeline, GitHub webhook triggers, and Nagios host definitions.

3. Master Distribution Matrix

Experiment	Member A (Shreya)	Member B (Khushboo)	Member C (Mahek)
EXP 01 Git	Steps 1–13 (Setup, first push); Steps 31, 32 (History, branches); Step 35 (Restore)	Steps 14–19 (Reviews branch); Step 26 (Merge); Step 33 (Soft reset)	Steps 20–25 (Discounts branch); Steps 27–30 (Merge conflict resolution); Step 34 (Revert)
EXP 02 Docker	Installation steps 1–6; Commands 1–9; Dockerfile and image build	Installation steps 1–6; Image build; Commands 10–17 including Linux container shell commands	Installation steps 1–6; Image build; Commands 18–25
EXP 03 Compose	`docker --version`; `mkdir` & `cd`; web & app services with `nginx.conf`; `down`, `up -d`, `config`	Database and phpMyAdmin services; `docker compose config`, `up -d`, `ps`, `logs`, `images`	Redis service; `docker compose stop`, `start`, `restart`, `down`, `down -v`, `config`
EXP 04 Jenkins CI	Procedure 1–3 (JDK verification, `jenkins.war`, unlock, plugins, admin setup, user creation)	Procedure 4 & 5 (Declarative Pipeline creation, execution, and console output)	Procedure 4 & 5 (Scripted Pipeline creation, execution, and console output)
EXP 05 Jenkins + GitHub	Part 1 (Verify Jenkins); Part 4 (Add `Jenkinsfile` to GitHub repository)	Part 2 (First pipeline job); Part 3 (Run locally); Webhook setup with ngrok	Part 5 (Connect Jenkins to GitHub via SCM); Part 6 (Verify GitHub-triggered build)
EXP 06 Jira	Steps 1–5 (Jira site, backlog, sprint, GitHub integration); Steps 9, 10 (GitHub Actions CI); Step 12 (Sprint completion)	Step 6 (Smart commit); Step 7 (Rule: In Review); Step 8 (Pull request); Step 11 (Review and merge PR)	Step 7 (Rule: Done); Step 8 (Review and merge PR); Step 11 (Second branch, merge conflict resolution)
EXP 07 Monitoring	Steps 1–4 (Nagios/Prometheus server container, admin password, web access); Step 7 (Verify connectivity)	Steps 5–6 (Install & configure NSClient++ on remote host)	Steps 8–9 (Add remote host definition, verify in web interface)

4. EXPERIMENT 01: Version Control Operations using Git

- **Aim:** To implement version control operations using Git, branch-based team collaboration, conflict resolution, and history manipulation.
- **LO Mapped:** LO2 (Perform version control operations using Git and apply collaboration workflows).
- **Work Division:**
  - **Member A (Shreya):** Steps 1–13, 31, 32, 35
  - **Member B (Khushboo):** Team step (Clone), Steps 14–19, 26, 33
  - **Member C (Mahek):** Team step (Clone), Steps 20–25, 27–30, 34

4.1 Member A (Shreya): Steps 1 to 13

Step 1 | Create Project Folder

```
mkdir devops-fullstack-app
cd devops-fullstack-app
```

Step 2 | Initialize Git Repository

```
git init
```

Step 3 | Configure Username

```
git config --global user.name "Shreya Khandelwal"
```

Step 4 | Configure Email

```
git config --global user.email "shreya.student@example.com"
```

Step 5 | Verify Configuration

```
git config --list
```

Step 6 | Create Project File

```
echo "ShopSphere E-Commerce Collaboration Project" > project.txt
```

Step 7 | Check Repository Status

```
git status
```

Step 8 | Stage Files

```
git add .
```

Step 9 | Create Initial Commit

```
git commit -m "Initial commit"
```

Step 10 | Rename Branch to Main

```
git branch -M main
```

Step 11 | Connect Remote Repository

```
git remote add origin https://github.com/<gh-user>/shopsphere-ecommerce.git
```

Step 12 | Verify Remote Repository

```
git remote -v
```

Step 13 | Push Main Branch

```
git push -u origin main
```

> **Hand-off:** Member A notifies Member B and Member C that `main` is published on GitHub. Member A invites B and C as Collaborators in GitHub Repository Settings.

Team Step | Members B and C | Clone Repository

```
git clone https://github.com/<gh-user>/shopsphere-ecommerce.git
cd shopsphere-ecommerce
```

(Members B and C configure their respective usernames and emails using Steps 3 and 4).

4.2 Member B (Khushboo): Steps 14 to 19 (Reviews Branch)

Step 14 | Create Resume/Product Reviews Feature Branch

```
git checkout -b feature-product-reviews
```

Step 15 | Modify Project File

```
echo "Product Reviews and Ratings Module" >> project.txt
```

Step 16 | Stage Changes

```
git add .
```

Step 17 | Commit Feature

```
git commit -m "Added product reviews module"
```

Step 18 | Push Feature Branch to GitHub

```
git push origin feature-product-reviews
```

Step 19 | Switch to Main Branch

```
git checkout main
```

4.3 Member C (Mahek): Steps 20 to 25 (Discounts Branch)

(Member C must branch from `main` before Member B's branch is merged into `main`).

Step 20 | Create Discounts Feature Branch

```
git checkout -b feature-discounts-coupons
```

Step 21 | Modify Project File

```
echo "Discounts and Coupon Codes Module" >> project.txt
```

Step 22 | Stage Changes

```
git add .
```

Step 23 | Commit Feature

```
git commit -m "Added discounts and coupons module"
```

Step 24 | Push Feature Branch to GitHub

```
git push origin feature-discounts-coupons
```

Step 25 | Switch to Main Branch

```
git checkout main
```

4.4 Merge, Conflict Resolution & Push: Steps 26 to 30

Step 26 | Member B (Khushboo) | Merge First Feature Branch

```
git merge feature-product-reviews
```

Team Step | Member B | Push Merged Main to GitHub

```
git push origin main
```

> **Hand-off:** Member B informs Member C that the first feature is merged and pushed.

Team Step | Member C (Mahek) | Pull Updated Main Branch

```
git checkout main
git pull origin main
```

Step 27 | Member C | Attempt Merge (Conflict Generation)

```
git merge feature-discounts-coupons
```

(Git detects concurrent modifications to `project.txt` and reports a `CONFLICT (content): Merge conflict in project.txt`).

Step 28 | Member C | Resolve Merge Conflict

Open `project.txt`. Remove conflict markers (`<<<<<<< HEAD`, `=======`, `>>>>>>> feature-discounts-coupons`) and retain both module entries:

```
ShopSphere E-Commerce Collaboration Project
Product Reviews and Ratings Module
Discounts and Coupon Codes Module
```

Stage the resolved file:

```
git add project.txt
```

Step 29 | Member C | Commit Merge Resolution

```
git commit -m "Resolved merge conflict"
```

Step 30 | Member C | Push Merged Main to GitHub

```
git push origin main
```

4.5 History, Rollback, Revert and Restore: Steps 31 to 35

Step 31 | Member A (Shreya) | View Commit History Graph

```
git pull origin main
git fetch --all
git log --oneline --graph --all
```

Step 32 | Member A | List Branches

```
git branch -a
```

Step 33 | Member B (Khushboo) | Soft Reset (Roll Back Last Commit)

```
git checkout -b practice-reset
echo "Temporary Reset Test" >> project.txt
git add .
git commit -m "Temporary commit for reset"
git reset --soft HEAD~1
git status
```

Cleanup after taking output screenshot:

```
git restore --staged .
git restore .
git checkout main
```

Step 34 | Member C (Mahek) | Revert Commit

```
git checkout -b practice-revert
echo "Temporary Revert Test" >> project.txt
git add .
git commit -m "Temporary commit for revert"
git log --oneline -n 2
git revert --no-edit HEAD
git checkout main
```

Step 35 | Member A (Shreya) | Restore Uncommitted Changes

```
git checkout main
echo "Unwanted accidental change" >> project.txt
git status
git restore .
git status
```

---

5. EXPERIMENT 02: Containerise the Application using Docker

- **Aim:** To containerise the full-stack web application using Docker and execute the 25 fundamental Docker commands.
- **LO Mapped:** LO4 (Containerize, orchestrate, and deploy applications using Docker).
- **Work Division:**
  - **All Members:** Installation steps 1–6, Image build & container run.
  - **Member A (Shreya):** Commands 1 to 9.
  - **Member B (Khushboo):** Commands 10 to 17 including Linux container shell commands.
  - **Member C (Mahek):** Commands 18 to 25.

5.1 Docker Installation & Verification (All Members)

1. Verify virtualization in Windows BIOS / Task Manager (`Virtualization: Enabled`).
2. Run `wsl --install` and restart computer if WSL 2 is missing.
3. Install and launch Docker Desktop.
4. Verify installation:

```
docker --version
docker run hello-world
```

5.2 Containerise the Application

Member A adds `backend/Dockerfile` :

```
FROM node:20-alpine AS runner
WORKDIR /app
RUN apk add --no-cache curl
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 5000
HEALTHCHECK --interval=15s --timeout=5s --retries=3 CMD curl -f http://localhost:5000/health
|| exit 1
CMD ["node", "src/server.js"]
```

All members pull and build the Docker image:

```
docker build -t shopsphere-backend:v1 .
docker run -d --name shopsphere-app -p 5000:5000 shopsphere-backend:v1
curl http://localhost:5000/health
```

5.3 Member A (Shreya): Commands 1 to 9

- **Command 1 | Docker Version:** `docker --version`
- **Command 2 | Search with Stars Filter:** `docker search ubuntu --filter stars=500`
- **Command 3 | Pull Image:** `docker pull ubuntu`
- **Command 4 | Interactive Container:** `docker run -it ubuntu /bin/bash` *(type `exit` to quit)*
- **Command 5 | List Running Containers:** `docker ps`
- **Command 6 | Run Named Container:** `docker run --name mycontainer -it ubuntu bash`
- **Command 7 | Stop Container:** `docker stop mycontainer`
- **Command 8 | Start Container:** `docker start mycontainer`
- **Command 9 | Inspect Container:**

```
docker inspect mycontainer
docker inspect shopsphere-app
```

5.4 Member B (Khushboo): Commands 10 to 17

- **Command 10 | List All Containers (Active & Stopped):** `docker ps -a`

- **Command 11 | Container Resource Stats:** `docker stats --no-stream`
- **Command 12 | Force Remove Container:** `docker rm -f mycontainer`
- **Command 13 | Run Nginx in Detached Mode:** `docker run -d --name web nginx`
- **Command Linux | Shell Operations Inside Container:**

```
docker exec -it shopsphere-app sh
ls
mkdir test && cd test && touch file1.txt && ls
exit
```

- **Command 14 | Pull Alpine Image:** `docker pull alpine`
- **Command 15 | Search Images on Docker Hub:** `docker search node`
- **Command 16 | Prune Dangling Images:** `docker image prune -f`
- **Command 17 | Container List Alias:** `docker container ls`

5.5 Member C (Mahek): Commands 18 to 25

- **Command 18 | System Information:** `docker info`
- **Command 19 | Image History and Layers:**

```
docker history ubuntu
docker history shopsphere-backend:v1
```

- **Command 20 | Disk Usage Breakdown:** `docker system df`
- **Command 21 | Inspect Docker Bridge Network:** `docker network inspect bridge`
- **Command 22 | Create Persistent Volume:** `docker volume create shopsphere-data`
- **Command 23 | Create User-Defined Network:** `docker network create shopsphere-net`
- **Command 24 | System Prune:** `docker system prune -f`
- **Command 25 | Rename Container:**

```
docker rename shopsphere-app shopsphere-backend-prod
```

---

6. EXPERIMENT 03: Multi-Service Application using Docker Compose

- **Aim:** To deploy multi-service applications using Docker Compose.
- **Architecture:** Web (Nginx on `8081`), Backend App (Node on `5000`), Database (MySQL on `3306` / PostgreSQL on `5432`), phpMyAdmin on `8082`, and Redis Cache on `6379`.
- **Work Division:**
  - **Member A (Shreya):** Web and App services (`compose-a` branch)
  - **Member B (Khushboo):** Database and phpMyAdmin services (`compose-b` branch)
  - **Member C (Mahek):** Redis cache and lifecycle commands (`compose-c` branch)

6.1 Member A (Shreya): Services Web and App

Create directory `compose-demo/` with `compose-demo/nginx.conf`:

```
server {
    listen 80;
    location / {
        proxy_pass http://app:5000;
    }
}
```

Create `compose-demo/docker-compose.yml`:

```
version: '3.8'
services:
  web:
    image: nginx
    ports:
      - "8081:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
  app:
    build:
      context: ..
      dockerfile: backend/Dockerfile
    environment:
      PORT: 5000
```

Execution:

```
cd compose-demo
docker compose up -d
docker compose config
```

Commit, push branch `compose-a`, and merge PR into `main`.

6.2 Member B (Khushboo): Services Database & phpMyAdmin

Pull `main` and append database services to `compose-demo/docker-compose.yml`:

```
  database:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: shopsphere_db
  phpmyadmin:
    image: phpmyadmin
    ports:
      - "8082:80"
    environment:
      PMA_HOST: database
```

Execution:

```
docker compose config
docker compose up -d
docker compose ps
docker compose logs database
docker compose images
```

Verify phpMyAdmin at `http://localhost:8082`. Commit, push branch `compose-b`, and merge PR into `main`.

6.3 Member C (Mahek): Redis Cache & Lifecycle Commands

Pull `main` and append Redis service to `compose-demo/docker-compose.yml`:

```
  redis:
    image: redis:alpine
```

Execution of full lifecycle:

```
docker compose config
docker compose up -d
docker compose ps
docker compose stop
docker compose start
docker compose restart
docker compose down
docker compose down -v
```

Commit, push branch `compose-c`, and merge PR into `main`.

7. EXPERIMENT 04: Continuous Integration with Jenkins (Declarative & Scripted Pipelines)

- **Aim:** Install Jenkins, configure role-based access, and implement CI pipelines using Declarative and Scripted syntax.
- **Work Division:**
  - **Member A (Shreya):** Jenkins installation, port `8080` firewall rule, user accounts for B and C.
  - **Member B (Khushboo):** Declarative Pipeline (`ShopSphere-Declarative`).
  - **Member C (Mahek):** Scripted Pipeline (`ShopSphere-Scripted`).

7.1 Member A (Shreya): Jenkins Server Setup

```
java --version
cd C:\jenkins
java -jar jenkins.war --httpPort=8080
```

Open `http://localhost:8080`, unlock with initial admin password, install suggested plugins, and create administrator credentials.

Enable Network Access for Team Members B & C:

```
ipconfig
New-NetFirewallRule -DisplayName "Jenkins 8080" -Direction Inbound -LocalPort 8080 -Protocol
TCP -Action Allow
```

In Jenkins: Navigate to **Manage Jenkins $\to$ Users $\to$ Create User**, and create accounts for `khushboo` and `mahek`.

7.2 Member B (Khushboo): Declarative Pipeline Job

1. Login to Jenkins at `http://<A-IP>:8080`.
2. Click **New Item** $\to$ Name: `ShopSphere-Declarative` $\to$ Select **Pipeline**.
3. Under Pipeline Script, enter:

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building ShopSphere E-Commerce Application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Running Automated API and Unit Tests...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying ShopSphere Containerized Stack...'
            }
        }
    }
}
```

4. Click **Build Now**, inspect **Console Output**, and verify `Finished: SUCCESS`.

7.3 Member C (Mahek): Scripted Pipeline Job

1. Login to Jenkins at `http://<A-IP>:8080`.
2. Click **New Item** $\to$ Name: `ShopSphere-Scripted` $\to$ Select **Pipeline**.
3. Under Pipeline Script, enter:

```
node {
    stage('Build') {
        echo 'Compiling ShopSphere Web Assets...'
    }
    stage('Test') {
        echo 'Executing Jest / PyTest Test Suites...'
    }
    stage('Deploy') {
        echo 'Deploying to Staging Environment...'
    }
}
```

4. Click **Build Now** and confirm `Finished: SUCCESS`.

---

8. EXPERIMENT 05: Automate Deployment using GitHub & Jenkins

- **Aim:** Automate CI/CD pipeline triggers from GitHub pushes via Webhooks and Pipeline script from SCM.
- **Work Division:**
  - **Member A (Shreya):** Create `Jenkinsfile` in GitHub repository root.
  - **Member B (Khushboo):** Create pipeline job, test locally, set up public ngrok webhook tunnel.

- **Member C (Mahek):** Configure GitHub SCM integration & verify automatic trigger on push.

8.1 Member A (Shreya): Add Jenkinsfile to GitHub Repository

In root of `shopsphere-ecommerce`, commit `Jenkinsfile`:

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building ShopSphere E-Commerce Production Bundle...'
            }
        }
        stage('Test') {
            steps {
                echo 'Executing Automated Tests...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying Containers with Docker Compose...'
            }
        }
    }
}
```

Push commit to `main`.

8.2 Member B (Khushboo): Webhook Setup

1. Create Pipeline job `ShopSphere-Demo` on Jenkins.
2. Launch public tunnel on Member A's machine:

   ```
   ngrok http 8080
   ```

3. In GitHub repo $\to$ **Settings $\to$ Webhooks $\to$ Add webhook**:
   - **Payload URL:** `https://<ngrok-id>.ngrok-free.app/github-webhook/`
   - **Content type:** `application/json`
   - **Events:** Just the push event.

8.3 Member C (Mahek): SCM Configuration & Automated Verification

1. In `ShopSphere-Demo` $\to$ **Configure**:
   - Under **Build Triggers**, check **GitHub hook trigger for GITScm polling**.
   - Under **Pipeline**, set Definition to **Pipeline script from SCM**.
   - SCM: **Git**, Repository URL: `https://github.com/<gh-user>/shopsphere-ecommerce.git`, Branch: `*/main`.
2. Push any change to GitHub $\to$ Observe Jenkins build start automatically with zero manual clicks!

---

9. EXPERIMENT 06: Agile Project Lifecycle using Jira with DevOps Integration

- **Aim:** Implement Agile Scrum sprint workflows, GitHub integration, Smart Commits, automation rules, and conflict resolution.
- **Work Items Used:**
  - `SHOP-4` (Task): Setup CI Pipeline with GitHub Actions (Assigned: Shreya)
  - `SHOP-5` (Story): Customer Reviews Feature (Assigned: Khushboo)
  - `SHOP-6` (Story): Coupon Discounts & API Tests (Assigned: Mahek)

9.1 Member A (Shreya): Jira Scrum Project & Sprint Setup

1. Sign up at atlassian.com, create Scrum space **ShopSphere Team**.
2. Board columns: `To Do`, `In Progress`, `In Review`, `Done`.
3. Create Backlog items (`SHOP-4`, `SHOP-5`, `SHOP-6`), assign story points, create **Sprint 1**, and click **Start Sprint**.
4. In Jira, install **GitHub for Jira** app and connect repository `shopsphere-ecommerce`.

9.2 Member B (Khushboo): Smart Commits & Automation Rule 1

1. Create branch and commit with Jira issue key:

```
git checkout -b SHOP-5-customer-reviews
echo "Customer Reviews component" >> README.md
git add .
git commit -m "SHOP-5 Add customer reviews feature description"
git push origin SHOP-5-customer-reviews
```

2. In Jira $\to$ **Project Settings $\to$ Automation $\to$ Create Rule**:
   - **Trigger:** Pull request created
   - **Action:** Transition issue to `In Review`
3. Create PR on GitHub $\to$ Verify `SHOP-5` moves automatically to `In Review`.

9.3 Member C (Mahek): Automation Rule 2 & PR Merge

1. In Jira $\to$ Create Automation Rule:
   - **Trigger:** Pull request merged
   - **Action:** Transition issue to `Done`
2. Approve and merge `SHOP-5` PR on GitHub $\to$ Verify `SHOP-5` moves automatically to `Done`.

9.4 Member A (Shreya): GitHub Actions CI Workflow

Add `.github/workflows/ci.yml`:

```
name: CI
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: cd backend && npm test
```

Commit and push on branch `SHOP-4-github-actions-ci`.

9.5 Member C (Mahek): Merge Conflict Resolution

1. Create `SHOP-6-coupon-tests` branch before A's PR is merged.
2. Concurrently append a test dependency in `requirements.txt` / `package.json`.
3. Member A merges `SHOP-4` into `main`.
4. Member C pulls `main`:

```
git fetch origin
git merge origin/main
```

5. Resolve conflict markers, commit resolution, and push:

```
git commit -m "SHOP-6 Resolve merge conflict"
git push origin SHOP-6-coupon-tests
```

6. Member B reviews and merges PR $\to$ `SHOP-6` transitions to `Done`.
7. Member A clicks **Complete Sprint** in Jira!

---

10. EXPERIMENT 07: Monitoring using Nagios & Prometheus

- **Aim:** Implement host and application monitoring using Nagios Core (Docker) and NSClient++ (Windows host agent), with Prometheus/Grafana integration.
- **Work Division:**
  - **Member A (Shreya):** Nagios Core / Prometheus server container on port `8090`.
  - **Member B (Khushboo):** Install and configure NSClient++ on remote Windows host.
  - **Member C (Mahek):** Register remote host definition in Nagios and verify in web dashboard.

10.1 Member A (Shreya): Nagios Server Container

```
docker pull jasonrivers/nagios:latest
docker run --name nagios4 -d -p 8090:80 jasonrivers/nagios:latest
docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin
```

Open `http://localhost:8090/nagios/` and login with `nagiosadmin`.

Open Inbound Firewall Rule:

```
New-NetFirewallRule -DisplayName "Nagios 8090" -Direction Inbound -LocalPort 8090 -Protocol
TCP -Action Allow
```

10.2 Member B (Khushboo): NSClient++ Agent Configuration

On Member B's PC:

```
Invoke-WebRequest -Uri "https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x
64.msi" -OutFile "NSCP.msi"
Start-Process msiexec.exe -ArgumentList '/i NSCP.msi ADDLOCAL=ALL MODULES=CheckExternalScrip
ts,NRPEServer /qn' -Wait
```

Edit `C:\Program Files\NSClient++\nsclient.ini`:

```
[/settings/default]
allowed hosts = 127.0.0.1,<A-IP>
```

Restart service & open firewall port `5666`:

```
Restart-Service nscp
New-NetFirewallRule -DisplayName "NRPE 5666" -Direction Inbound -LocalPort 5666 -Protocol TC
P -Action Allow
```

10.3 Member C (Mahek): Host Registration & Verification

Inside Nagios container, register Member B's host in `/opt/nagios/etc/servers/ShopSphereHost001.cfg`:

```
define host {
    use         windows-server
    host_name   ShopSphere_Host_001
    alias       ShopSphere Remote Host 001
    address     <B-IP>
    register    1
}

define service {
    host_name               ShopSphere_Host_001
    service_description     PING
    check_command           check_ping!100.0,20%!500.0,60%
    max_check_attempts      2
    check_interval          2
    retry_interval          2
    check_period            24x7
    notification_interval   2
    notification_period     24x7
    notifications_enabled   1
    register                1
}
```

Verify and restart Nagios:

```
docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg
docker restart nagios4
```

Open `http://<A-IP>:8090/nagios/` $\to$ **Hosts** $\to$ Confirm `ShopSphere_Host_001` is **UP** and **PING** service is **OK**!

---

11. Order of Execution and Hand-Offs

Sequence	Task	Dependencies	Acting Member
1	GitHub accounts created; Git, Node, Docker installed	None	All Members
2	**EXP 01 Git:** Steps 1–13 (Init, first push)	None	Member A (Shreya)
3	**EXP 01 Git:** Clone repository, create feature branches	Task 2 (Main pushed)	Members B & C
4	**EXP 01 Git:** Merge branch 1, generate and resolve conflict, history inspection	Task 3	B, then C, then A
5	**EXP 06 Jira:** Space creation, backlog, Sprint 1 setup, GitHub connection	Task 2 (Repo exists)	Member A (Shreya)
6	**EXP 02 Docker:** Dockerfiles created, images built, 25 commands executed	Task 4	A (1–9), B (10–17), C (18–25)
7	**EXP 03 Compose:** Multi-service composition (`compose-a`, `compose-b`, `compose-c`)	Task 6 (Docker images)	A, then B, then C
8	**EXP 04 Jenkins:** Server installation, Declarative & Scripted pipelines	Task 2 (JDK & network)	A (Install), B (Declarative), C (Scripted)
9	**EXP 05 Jenkins + GitHub:** Webhook setup via ngrok, automated trigger on push	Task 8	A (File), B (Tunnel), C (Trigger)
10	**EXP 06 Jira & Actions:** Smart commits, PR automation rules, Actions CI run, Complete Sprint	Tasks 5 & 9	B (PR), C (Merge), A (Actions & Finish)
11	**EXP 07 Monitoring:** Nagios/Prometheus container setup, NSClient++ agent, Host verification	Task 6 (Docker host)	A (Server), B (Agent), C (Host Config)

---

12. Final Integrated Demonstration Checklist

- ☑ **Jira Agile Lifecycle:** Issue key (`SHOP-5`) included in Git branch and commit message; PR automatically shifts ticket to `In Review`, and merge shifts to `Done`.
- ☑ **GitHub Actions CI:** Automated test suite runs green upon code push to `main`.
- ☑ **Jenkins Continuous Deployment:** Webhook triggers automated `ShopSphere-Demo` pipeline from GitHub SCM.

- ☑ **Docker Compose Stack:** `docker compose up -d` brings up Frontend, Backend, Database, Redis, and Nginx.
- ☑ **Monitoring Active:** Nagios shows `ShopSphere_Host_001` as **UP** with active PING status, and Prometheus `/metrics` exports real-time shopping traffic.

---

13. Common Troubleshooting & Fixes

Problem Encountered	Root Cause	Solution
Git merge conflict treated as binary file	Windows PowerShell 5.1 default `>` redirects save as UTF-16.	Re-save file with UTF-8 encoding in VS Code or run echo commands in Git Bash / PowerShell 7.
Docker daemon connection error	Docker Desktop engine is stopped.	Start Docker Desktop application and wait until engine indicator turns green.
Port already in use (`5000`, `8080`, `8090`)	Background service or dangling container occupying port.	Identify process using `netstat -ano | findstr :<port>` or run `docker rm -f <container-name>`.
Members cannot access Jenkins or Nagios	Windows Firewall blocking inbound LAN traffic on Member A's PC.	Run `New-NetFirewallRule -DisplayName "Port Rule" -Direction Inbound -LocalPort 8080 -Protocol TCP -Action Allow` in admin PowerShell.
GitHub Webhook red cross error	ngrok tunnel disconnected or URL changed.	Re-launch `ngrok http 8080` and update Payload URL in GitHub repository settings.
Jira Automation rule not firing	Branch or PR missing exact issue key format.	Ensure branch or commit title strictly contains key (e.g. `SHOP-5`).